APOSTILA DE REVISÃO V1

HTML 5 & CSS 3

Módulo 12: Grid CSS

FORMULA WEB PRO-MAX

O Que São Grids?

Grids em CSS são sistemas de layout que permitem organizar elementos HTML em uma grade bidimensional. Isso simplifica o design de páginas da web, tornando-as mais previsíveis e flexíveis. Com grids, você pode criar layouts complexos e responsivos com facilidade, definindo um container de grid, linhas, colunas e posicionando os elementos dentro dessa grade. É uma ferramenta fundamental para o design web moderno.

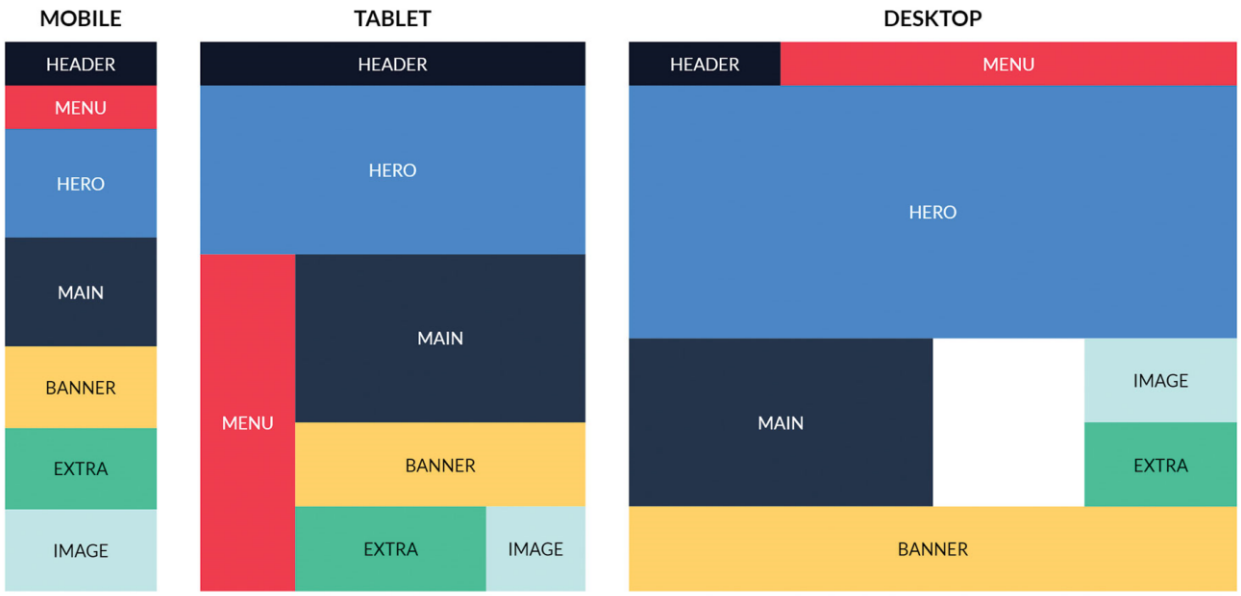

Figura 1: Exemplo de uso de grids.

Grids CSS e Flexbox são duas técnicas de layout em CSS que têm semelhanças, mas também diferenças importantes. Veja abaixo suas semelhanças, diferenças e exemplos de casos de uso para cada um.

Semelhanças:

1. **Layout em Duas Dimensões:** Ambos Grids CSS e Flexbox permitem criar layouts em duas dimensões (linhas e colunas no caso de grids, e eixos principal e transversal no caso de flexbox).

2. **Controle de Espaçamento:** Ambos oferecem controle sobre o espaçamento e o alinhamento dos elementos.

Diferenças:

1. **Orientação:** A maior diferença entre eles é a orientação. Flexbox é principalmente orientado para layouts unidimensionais, enquanto Grids CSS é orientado para layouts bidimensionais.
2. **Controle de Elementos:** Com Flexbox, você controla o posicionamento e o espaçamento dos elementos dentro de um único eixo (linha ou coluna). Em Grids CSS, você controla o posicionamento e o espaçamento em duas direções, permitindo layouts mais complexos.
3. **Uso de Espaço Vazio:** Grids são melhores para preencher espaços vazios em layouts, enquanto Flexbox é mais adequado para alinhar elementos em um único eixo, deixando espaço vazio no outro eixo.

Exemplos de Casos de Uso:

1. **Flexbox é ideal para:**
   - Criar barras de navegação horizontais ou verticais.
   - Alinhar itens em um único eixo, como uma lista de itens em uma barra lateral.
   - Criar layouts flexíveis de cabeçalho e rodapé.
2. **Grids CSS são ideais para:**
   - Layouts de grade complexos, como layouts de revistas com várias colunas e linhas.
   - Organizar elementos em várias colunas e linhas, como em uma galeria de imagens.
   - Criar layouts responsivos com facilidade.

Em resumo, escolha Flexbox quando precisar alinhar elementos em um único eixo ou criar layouts unidimensionais. Escolha Grids CSS quando precisar criar layouts bidimensionais complexos ou organizar elementos em várias colunas e linhas. Em muitos casos, você pode até mesmo combiná-los para obter o melhor dos dois mundos em termos de controle de layout.

Modelo de Colunas e Linhas Grid

Os modelos de colunas (grid-template-columns) e modelos de linhas(grid-template-rows) grid são usadas para definir a estrutura de linhas e colunas em um layout de grid CSS.

1. grid-template-columns: Esta propriedade é usada para definir a estrutura das colunas em um layout de grid. Ela especifica quantas colunas você deseja em seu grid e como essas colunas devem ser dimensionadas. Você pode definir colunas de várias maneiras:

- **Comprimentos fixos:** Você pode especificar larguras fixas para as colunas usando valores como 100px, 200px, etc.
- **Frações (fr):** Você pode usar a unidade fr (fração) para distribuir o espaço disponível entre as colunas de forma igual ou em proporções específicas.
- **Porcentagens:** Você pode definir as colunas em relação à largura do contêiner pai usando porcentagens, como 25%, 50%, etc.
- **Auto:** Você pode permitir que as colunas tenham um tamanho automático, com base no conteúdo dentro delas.

2. grid-template-rows: Esta propriedade é usada de maneira semelhante à grid-template-columns, mas define a estrutura das linhas em um layout de grid. Você pode especificar a altura das linhas de maneira semelhante às colunas, usando valores fixos, frações (fr), porcentagens ou tamanho automático.

Juntas, grid-template-columns e grid-template-rows permitem que você crie layouts complexos e flexíveis em grade CSS, controlando o número, o tamanho e a disposição das colunas e linhas em seu design. Essas propriedades são fundamentais para o uso eficiente de CSS Grid para criar layouts responsivos e dinâmicos.

Abrangimento de Colunas e Linhas Grid

EM Grids CSS nós podemos fazer com que um item dentro do grid abranja mais que uma coluna ou mais que uma linha, e para isso nós podemos usar as seguintes propriedades:

grid-column: 1 / 3;

grid-column-start: 3;

grid-column-end: 4;

grid-row: 1 / span 3;

grid-row-start: 2;

grid-row-end: span 4;

Vamos analisar cada uma delas:

1. **grid-column: 1 / 3:** define a posição inicial e final de um item de grid ao longo do eixo das colunas. Neste caso, o item começa na primeira coluna e se estende até a terceira coluna. Isso significa que o item ocupa as colunas 1 e 2, mas não a 3.
2. **grid-column-start: 3**: define apenas a posição inicial de um item de grid ao longo do eixo das colunas, sendo assim o item irá começar na coluna 3.
3. **grid-column-end: 3**: define apenas a posição final de um item de grid ao longo do eixo das colunas, neste caso o item irá ocupar a coluna 1 e 2.

4. **grid-row: 1 / span 3**: define a posição inicial e final de um item de grid ao longo do eixo das linhas. O item começa na primeira linha (linha 1) e se estende por 3 linhas adicionais (devido a span 3). Portanto, o item ocupa as linhas 1, 2 e 3.
5. **grid-row-start: 2**: define apenas a posição inicial de um item de grid ao longo do eixo das linhas, sendo assim o item irá começar na linha 2.
6. **grid-row-end: span 4**: define apenas a posição final de um item de grid ao longo do eixo das linhas, usando o span e assumindo que elemento começa primeira linha ela irá ocupar as linhas 1, 2, 3 e 4.

Em resumo, as propriedades grid-column e grid-row definem inicialmente a posição e a extensão do item de grid, mas as propriedades subsequentes grid-column-start, grid-column-end, grid-row-start e grid-row-end substituem essas configurações para alterar a posição das linhas do item dentro do grid. Certifique-se de que a ordem e os valores das propriedades sejam ajustados conforme necessário para obter o posicionamento desejado de elementos em seu layout de grid.

Auto-fit e minmax

A função auto-fit é usada em conjunto com a propriedade grid-template-columns (ou grid-template-rows) para criar um número dinâmico de colunas (ou linhas) em um layout de grid CSS. Ela ajusta automaticamente o número de colunas (ou linhas) com base no espaço disponível no contêiner. Exemplo:

```
grid-template-columns: repeat(auto-fit, minmax(100px, 1fr));
```

Neste exemplo, estamos usando auto-fit com minmax() para criar um número flexível de colunas. As colunas terão um tamanho mínimo de 100 pixels e um tamanho máximo de 1fr. O auto-fit ajustará o número de colunas com base no espaço disponível no contêiner.

Alinhamento em Grids CSS

O alinhamento desempenha um papel fundamental na criação de layouts atraentes e funcionais. Aqui estão alguns conceitos importantes de alinhamento em grids CSS:

1. **Alinhamento de itens no grid:** justify-items e align-items são propriedades que controlam o alinhamento horizontal e vertical dos itens do grid dentro de suas células. Por exemplo:

```
.grid-container {
        display: grid;
        justify-items: center; /* Alinha itens horizontalmente no centro */
        align-items: center; /* Alinha itens verticalmente no centro */
}
```

2. **Alinhamento de conteúdo no grid:** Você pode usar justify-content e align-content para alinhar o conteúdo do grid em relação ao próprio grid. Isso é útil quando o grid é maior que a área ocupada pelos itens. Por exemplo:

```
.grid-container {
  display: grid;
  grid-template-rows: 100px 100px;
  grid-template-columns: 100px 100px;
  justify-content: center; /* Alinha o conteúdo horizontalmente no centro */
  align-content: center; /* Alinha o conteúdo verticalmente no centro */
}
```

3. **Alinhamento de itens individuais:** Você pode controlar o alinhamento de itens específicos dentro do grid usando as propriedades justify-self e align-self. Isso permite que você ajuste o alinhamento de itens individualmente, ignorando as configurações globais. Por exemplo:

```
.grid-item {
    justify-self: start; /* Alinha o item à esquerda dentro de sua célula */
    align-self: end; /* Alinha o item na parte inferior dentro de sua célula */
}
```

4. **Espaçamento entre itens:**
   Para controlar o espaçamento entre os itens do grid, você pode usar grid-gap para definir espaçamentos tanto na direção horizontal quanto vertical. Também pode usar grid-row-gap e grid-column-gap para espaçamento específico nas linhas ou colunas.

```
.grid-container {
        display: grid;
        gap: 10px; /* Espaçamento entre todos os itens */
        row-gap: 5px; /* Espaçamento entre as linhas */
        column-gap: 15px; /* Espaçamento entre as colunas */
}
```

Áreas de modelo de Grid

Área de modelo de grid definida usando a propriedade css grid-template-areas permite criar layouts de grid de forma declarativa e visual, atribuindo nomes a áreas específicas da grade e, em seguida, associando elementos HTML a essas áreas por meio de seletores CSS. Isso torna a criação e manutenção de layouts complexos mais intuitiva e legível.

A propriedade grid-template-areas permite que você defina a estrutura da grade usando uma sintaxe de grade nomeada, onde cada área é identificada por um nome (uma sequência de texto) e é especificada dentro de um modelo de grade. Aqui está um exemplo básico:

```
grid-template-areas:
  "header header header"
  "sidebar main main"
  "footer footer footer";
```

Neste exemplo, estamos definindo um layout de grade com três linhas e três colunas. Cada célula da grade é nomeada com um nome (por exemplo, "header", "sidebar", "main", "footer"). Agora, você pode associar elementos HTML a essas áreas usando seletores CSS.

```
.header {
        grid-area: header;
}
.sidebar {
        grid-area: sidebar;
}
.main {
        grid-area: main;
}
.footer {
        grid-area: footer;
}
```

Com essa configuração, os elementos HTML com as classes correspondentes serão automaticamente colocados nas áreas apropriadas da grade. O Grid Layout cuida do posicionamento e dimensionamento dos elementos de acordo com o modelo de grade que você definiu. A principal vantagem do grid-template-areas é a capacidade de criar layouts visuais e entender facilmente a estrutura do layout apenas olhando para o código CSS. Isso é especialmente útil em equipes de desenvolvimento onde a legibilidade e a manutenção do código são importantes.

Além disso, grid-template-areas é útil para criar layouts responsivos, pois você pode reorganizar as áreas da grade para se adaptar a diferentes tamanhos de tela apenas ajustando as definições de grade, mantendo os seletores CSS intactos.

Em resumo, grid-template-areas é uma propriedade do CSS Grid que permite criar layouts de grid visualmente intuitivos, atribuindo nomes a áreas da grade e associando elementos a essas áreas usando seletores CSS, tornando a criação e manutenção de layouts complexos mais eficientes e compreensíveis.